CS214 Lab Week 4

Complexity Analysis

Algorithms in everyday life

= Priority queue + optimisation algorithm + magic

About Your Tutor (For the new faces)

- Pranav
- Software Engineering Student
- Bored with web development, trying to learn other things.
- **Email: s11171153@student.usp.ac.fj**

Getting to know you . . .

- We're stuck here for many weeks so might as well.
- Name ? Programme ?
- Something, anything, you want to say about software or algorithms ☺

Some Expectations

- Please bring ID cards for attendance (more convenient)
- Don’t be *unreasonably* late
- Ask ask ask a lot of questions

Coming up next

1. Recap of Binary Search algorithm
2. Complexity analysis of two algorithms
3. Complexity analysis of Fibonacci
4. Assignment helping time

Basically . . .

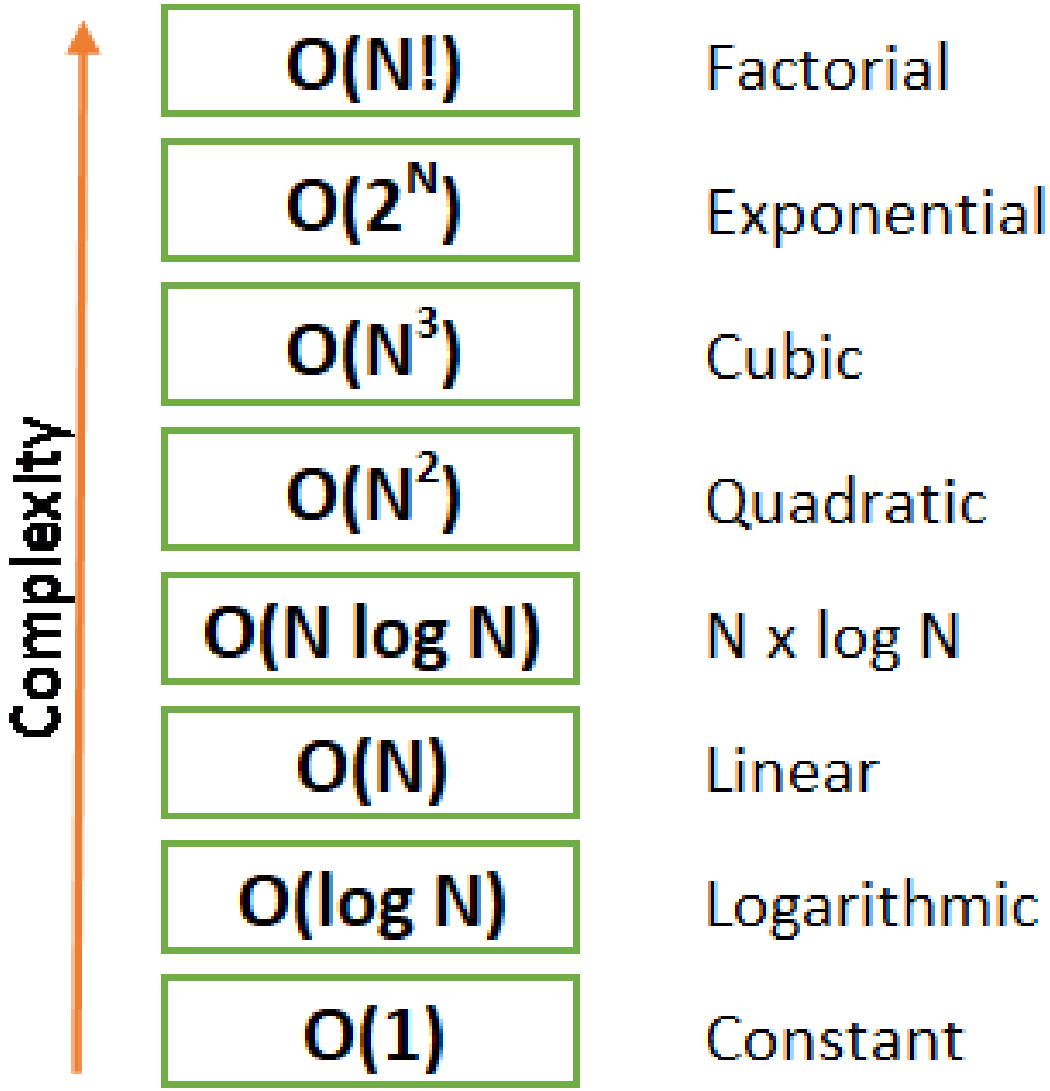
- Time Complexity - the measurement of the **<u>fundamental operations</u>** which algorithm carries out on a data of size n

Best Case	Average Case	Worst Case
Smallest number of operations which the algorithm will have to carry out on a data file of fixed size n	*Average of algorithm complexities* on data files of size n. <u>Can use Expected Value</u>	*Greatest number of operations* which the algorithm will have to carry out on a data file of fixed size

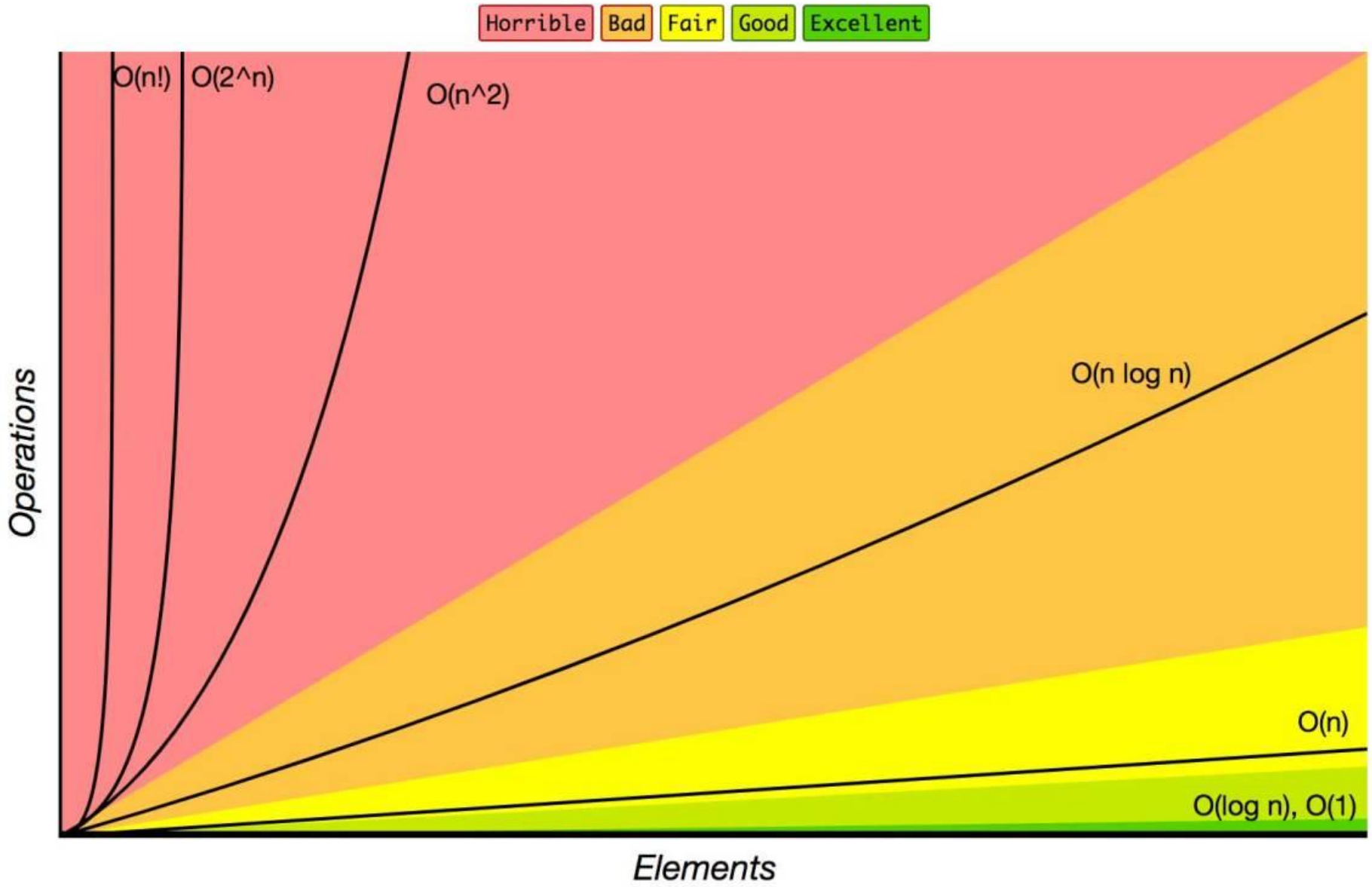
- Consider: When does each case occur? What is the operations for any size n?
- Best = Average = Worst, it is EVERY CASE TIME COMPLEXITY

BIG RULE for BIG O

Take the highest order variable!



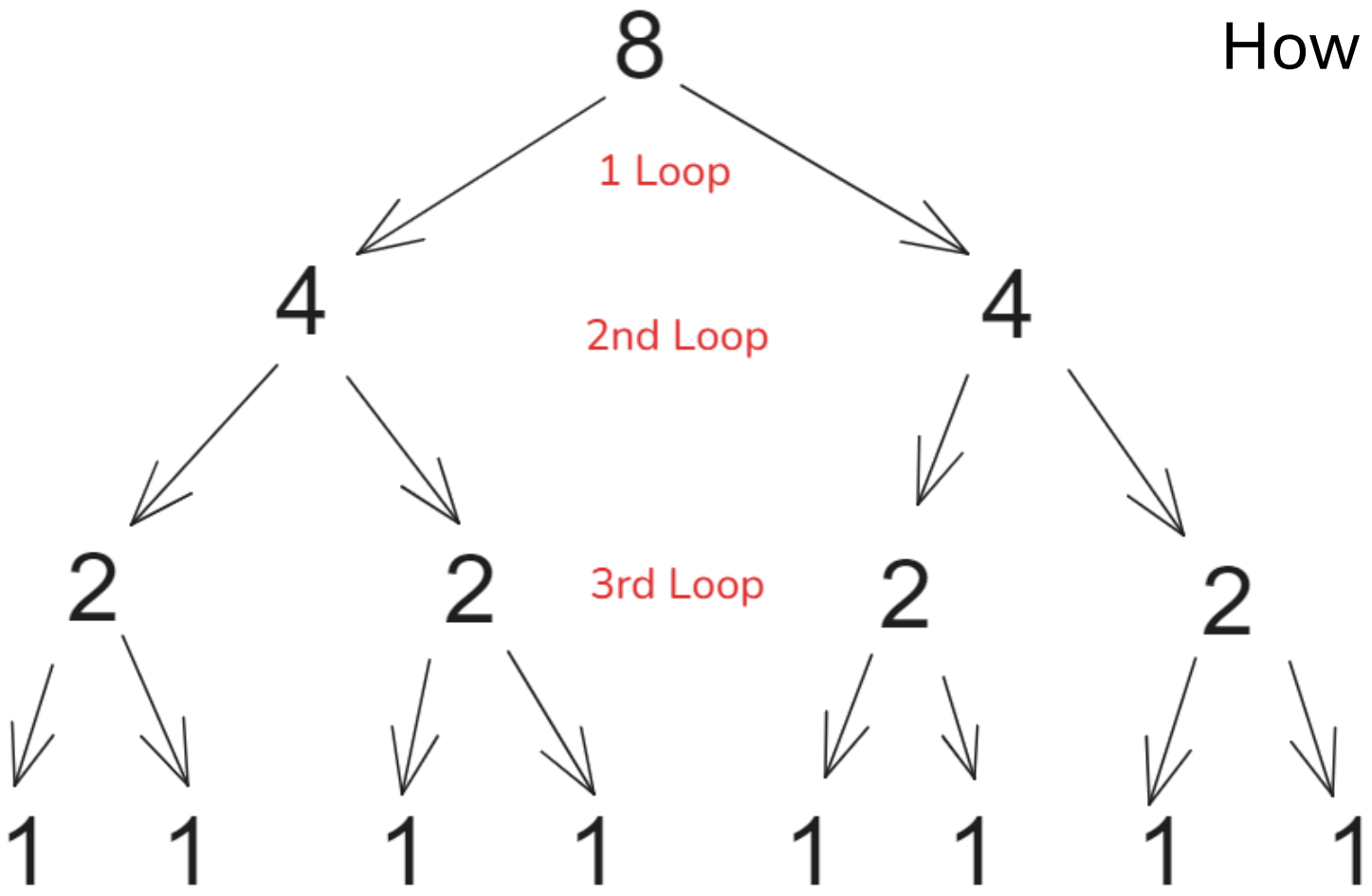
Big-O Complexity Chart



Binary Search Analysis

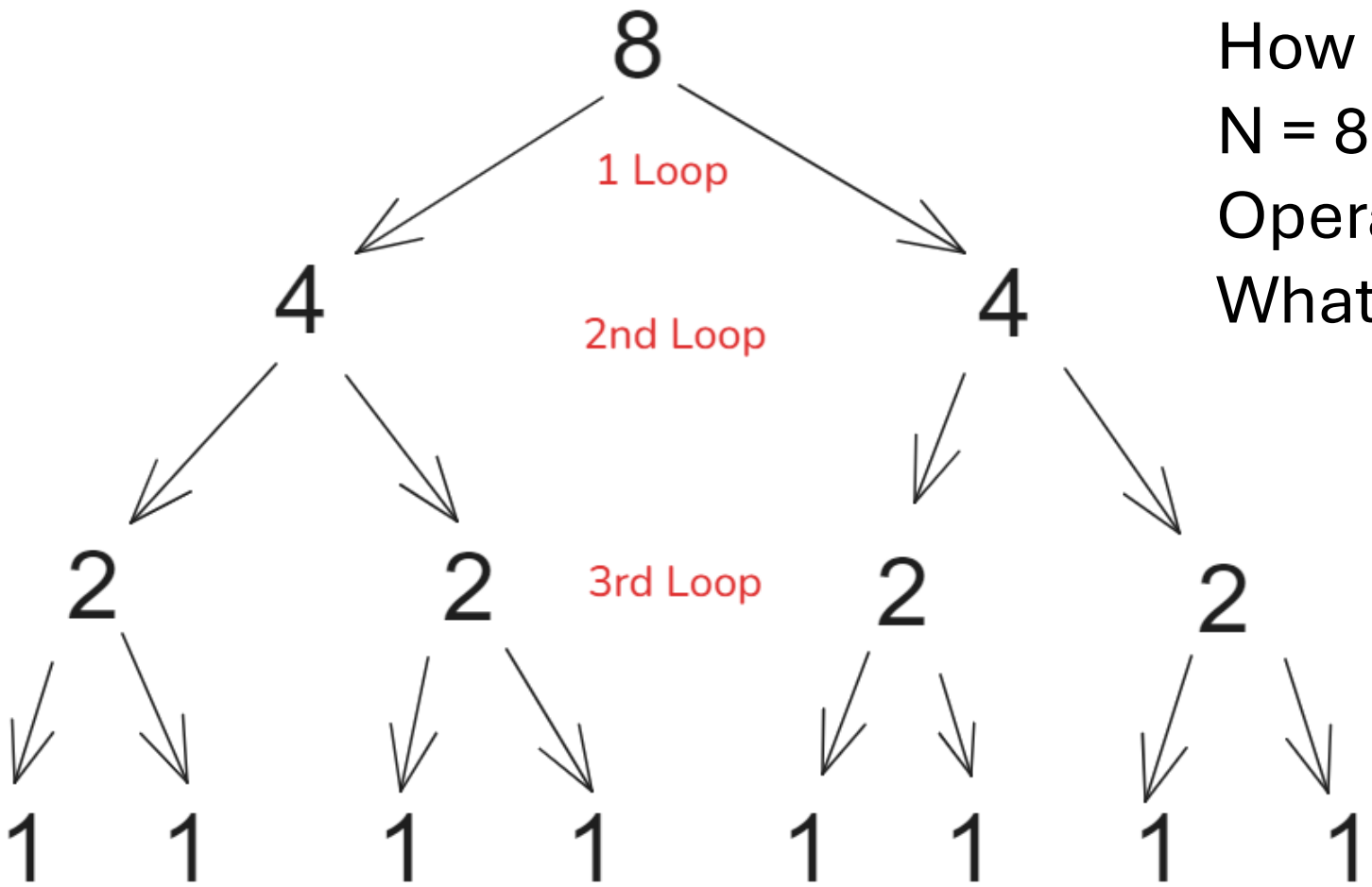
	Best Case	Average Case	Worst Case
Time Complexity	O (1)	O (log n)	O (log n)
Case	Found on first search	Found in any search or no search	Found in last or no search
When	Middle element	All element and no element	First, last and no element
Easy way	1 operation	Find expected value. Solve geometric sequence	Solve derived arithmetic sequence

Binary Search Worst Case Made Easier



How many operations?

Binary Search Worst Case Made Easier



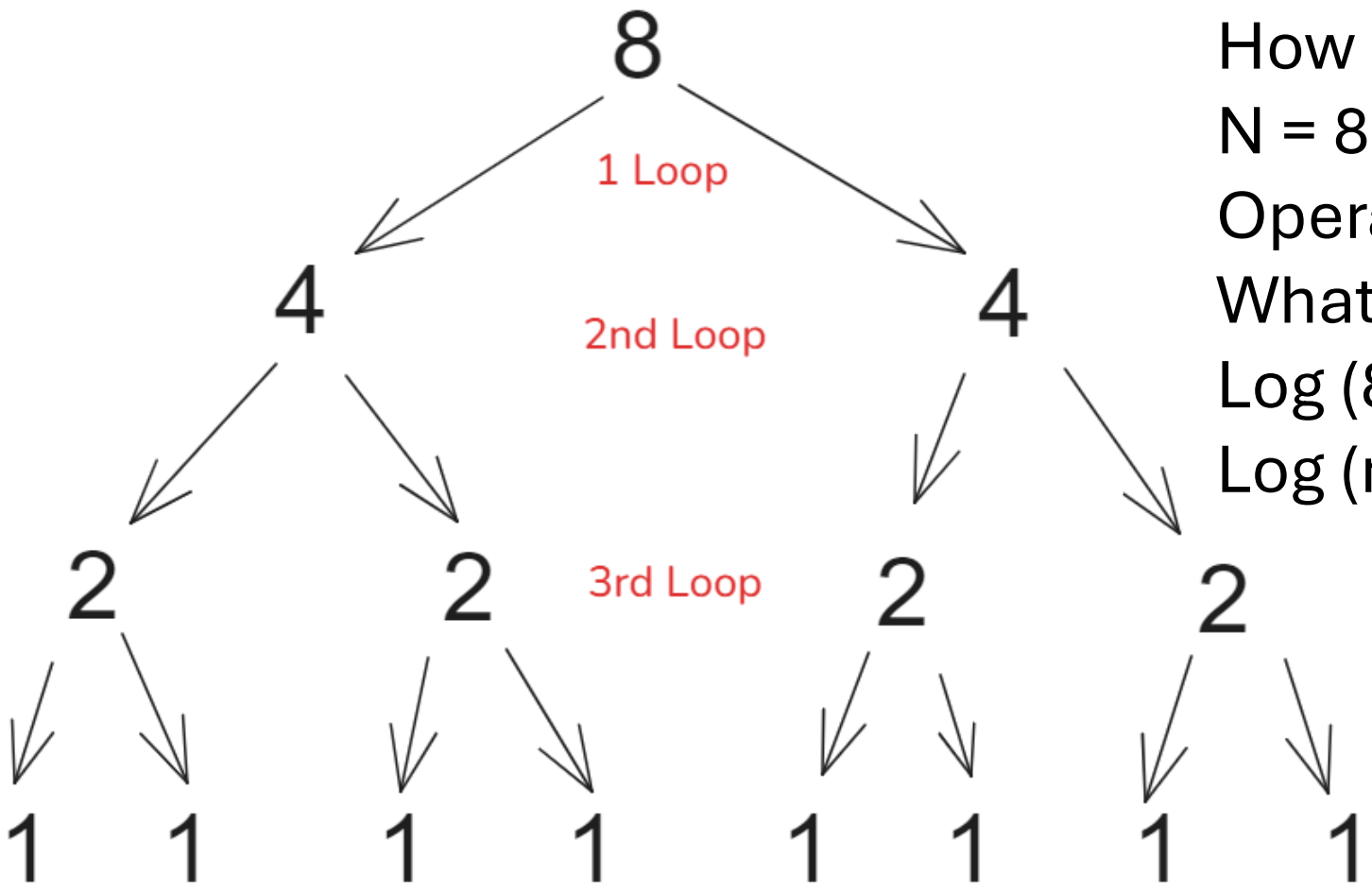
How many operations?

N = 8

Operations = 3

What's the deal?

Binary Search Worst Case Made Easier



How many operations?

N = 8

Operations = 3

What's the deal?

Log (8) = 3

Log (n) !!!!!

Time to flex our skills

A. Find the best, worst and average time complexity of following code:

```
int a = 0;
for (i = 0; i < N; i++) {
    for (j = N; j > i; j--) {
        a = a + i + j;
    }
}
```

When does best, average and worst time occur?

A. Find the best, worst and average time complexity of following code:

```
int a = 0;
for (i = 0; i < N; i++) {
   for (j = N; j > i; j--) {Inner Loop
      a = a + i + j;
   }
}
```

Tip: Ignore constants, its all about the loops

<u>Count your loops! Find pattern</u>

A. Find the best, worst and average time complexity of following code:

```
int a = 0;
for (i = 0; i < N; i++) {
   for (j = N; j > i; j--) {Inner Loop
      a = a + i + j;
   }
}
```

Count the inner loops for Size N

Inner Loop: Runs backwards, N to 0 times....

How can we represent this?

A. Find the best, worst and average time complexity of following code:

```
int a = 0;
for (i = 0; i < N; i++) {
   for (j = N; j > i; j--) {Inner Loop
      a = a + i + j;
   }
}
```

Count the loops for Size N

Inner Loop: Runs backwards, N to 0 times....

N + (N-1) + (N-2) .... + 1 + 0

A. Find the best, worst and average time complexity of following code:

```
int a = 0;
for (i = 0; i < N; i++) {Outer loop also run N times!
   for (j = N; j > i; j--) {Inner Loop run N times!
      a = a + i + j;        Thus N^2
   }
}
```

Count the inner loops for Size N

Inner Loop: Runs backwards, N to 0 times....

N + (N-1) + (N-2) .... + 1 + 0

= N*(N+1)/2

= O(N^2)

B. Find the Big O order of following code.

```
int i, j, k = 0;
for (i = n / 2; i <= n; i++) {
    for (j = 2; j <= n; j = j * 2) {
        k = k + n / 2;
    }
}
```

B. Find the Big O order of following code.

```
int i, j, k = 0;
for (i = n / 2; i <= n; i++) { Outer Loop
   for (j = 2; j <= n; j = j * 2) { Inner Loop
      k = k + n / 2;
   }
}
```

Outer Loop runs N/2 times

Inner loop: *Observe* $2, \quad 4, \quad 8, \quad 16 \ldots\ldots N$

$2^1, \quad 2^2, \quad 2^3, \quad 2^4 \ldots [\, 2^x = N \,]$

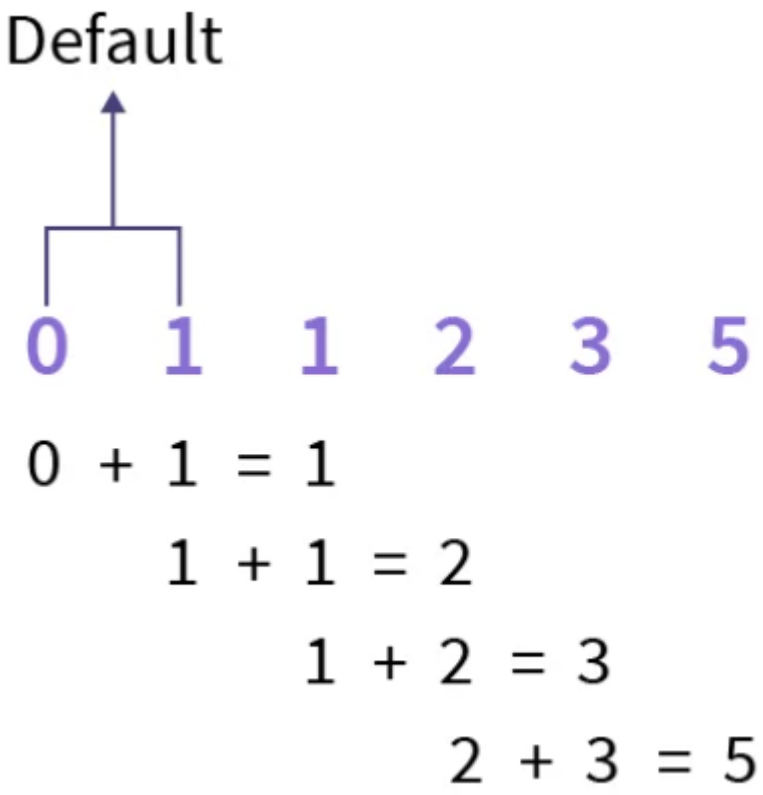
$\log_2 (2^x) = \log_2 N$

Inner Loop Runs $\log_2 N$ times

Time complexity is N/2 * log2 N

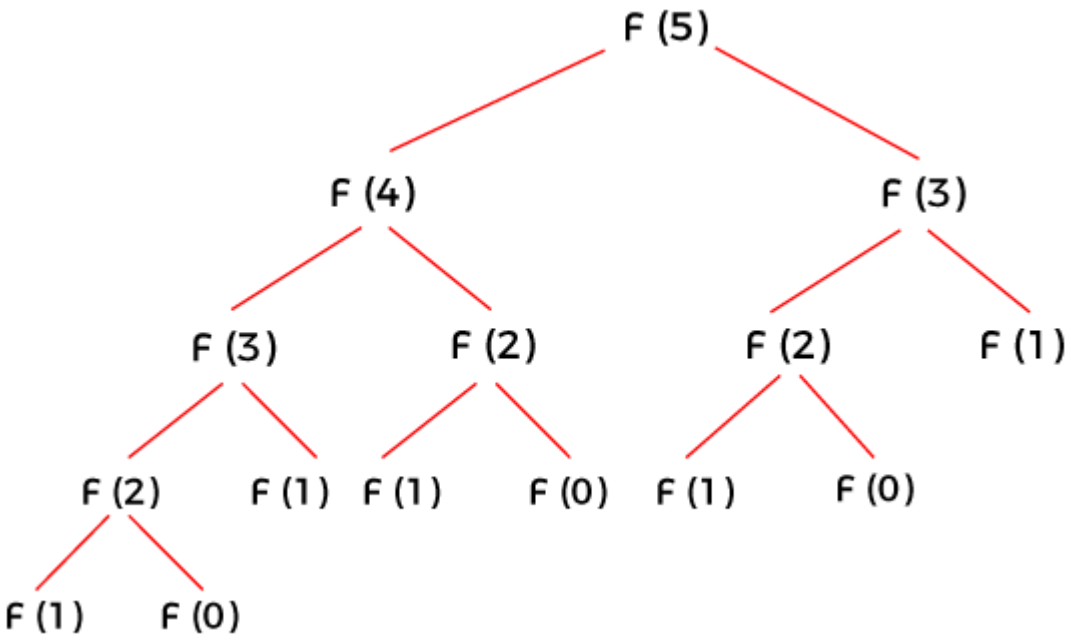
= O(N log2 N)

Fibonacci Time Complexity



- Recursive: O(2^n)
- Iterative: O (n)
- Dynamic Programming: O(n)

Fibonacci Time Complexity



- Dynamic Programming: O(n)
- Just remember previously calculated values silly computer. Don’t calculate it again.

Time complexity at your fingertips

- Take highest order variable in Big O
- Examine the loops
- Find a pattern
- Generalise relative to N

Any Assignment Clarifications?